Artificial Intelligence COMP9414 26T2

M11A, H18A tutorial

08/06/2026 Monday

Content of today:

1. A little bit about Joffrey
2. Introduction to marks and this tutorial
3. python introduction
4. Search algorithm
5. Q & A

Joffrey Ji (z5450981@ad.unsw.edu.au)

A little bit about Joffrey

1st year PhD,

interesting with: LLM, Epistemic Model, Planning agent, culture, Building Environment

Graduated at 25T2 UNSW
Master of IT (special in AI)
Final Thesis is about Reinforcement Learning, Robotic, VLM, UAV

Being Tutor for more than 2 years:

COMP9414 Artificial Intelligence:	25T2 - now
COMP9814/3411 Advanced Artificial Intelligence :	25T3
COMP9020 Foundation of Computer Sci.:	24T1 - 25T3
COMP9024 Data Structure and Algorithm:	24T2 – now
COMP6080 Web Front-End Programming:	26T1

Grading Structure

Assignment_1 = mark for Assignment 1 (out of 21%)
Assignment_2 = mark for Assignment 2 (out of 20%)
Encourage_mark = mark for Attendance + week1 quiz (out of 9%)
Final_exam = mark for final exam (out of 50%)
Initialize final_grade = 0

if (**Final exam** >= 40%):
    final grade = Assignment_1 + Assignment_2 + Encourage_mark + Final_exam
    if **final_grade** >= 50:
        return **pass**!!!
else:
    return **not pass**…

Discussion for A1 (21%) and A2 (20%)

For Assignment 1 and Assignment 2, your mark will be given by Joffrey, depends on your code, output, and conversation.

- 15-18 minutes each person
- Will be Scheduled ahead through moodle (A1 submission deadline will be at wk5 discussion on wk6-wk7, A2 submission deadline will be at wk9 discussion on wk10-wk11)
- Question will not be complete same, but depends on your code
- Have to download the code during the discussion, not before.
- But you can write a script before discussion.

Encourage mark for tutorial (9% in total)

For attendance and interaction, here's the breakdown:

- **5%** will come from Week 1–5 (1% for each tutorial, week1's quiz included).
- **4%** will come from Week 7–10.

That means you are expected to attend every tutorial. I will do my best to prepare useful content, so your time here won't be wasted. During class, please try to actively engage and work on the activities related to our tutorial.

YouTube bilibili

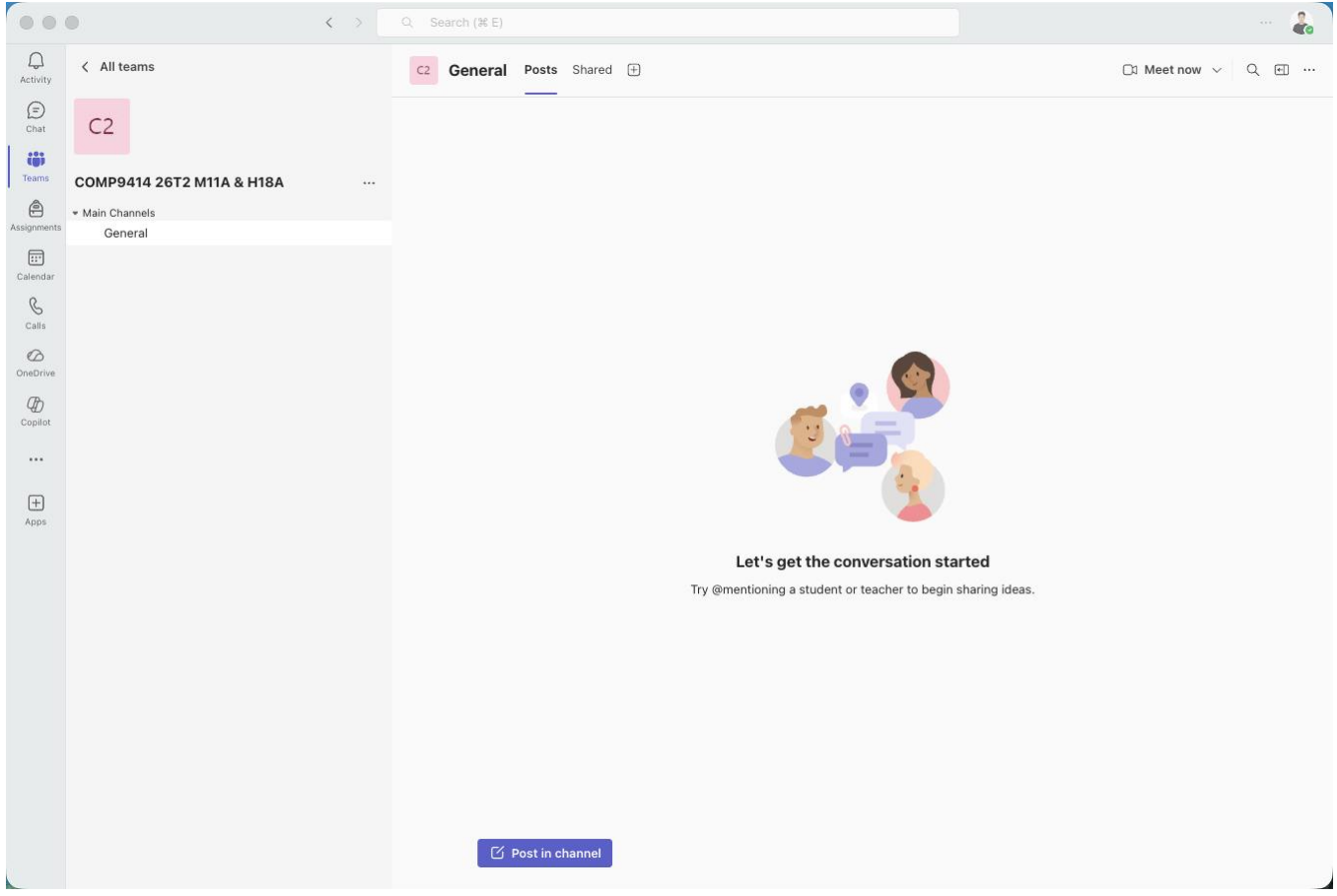
TikTok

UNSW SYDNEY

What will be in our tutorial?

- A summary of each lecture's content for last week.
- Tutorial experiments
- Selected extension exercises.
- Q&A session. Please send any questions you have, to Fourm or my email (z5450981@ad.unsw.edu.au) at anytime.

[General | COMP9414 26T2 M11A & H18A]



UNSW SYDNEY

About Jupyter Notebook and Python

Depth-First Search (DFS) Stack

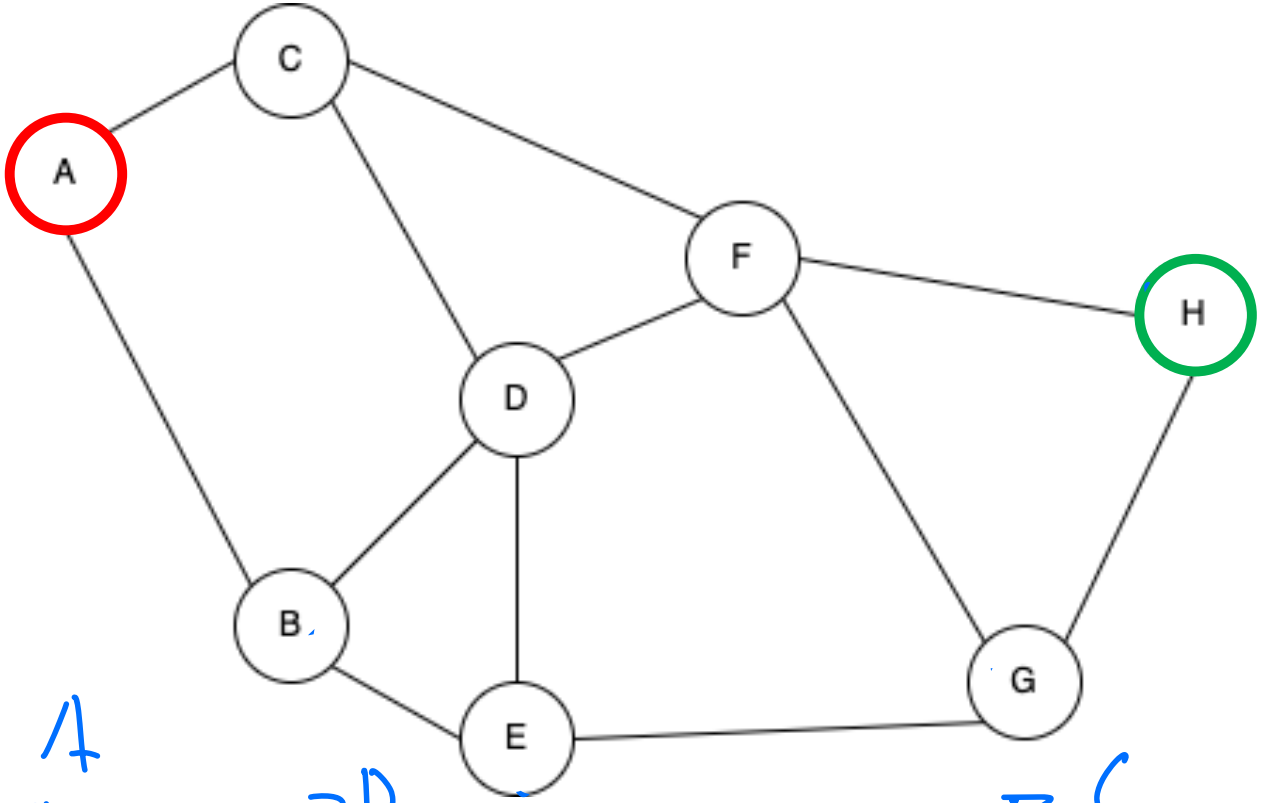
Depth-First Search (DFS) is an algorithm used for traversing or searching tree or graph data structures. The algorithm starts at the root node and explores as far as possible along each branch before backtracking. DFS uses a stack to keep track of the nodes to be visited next.

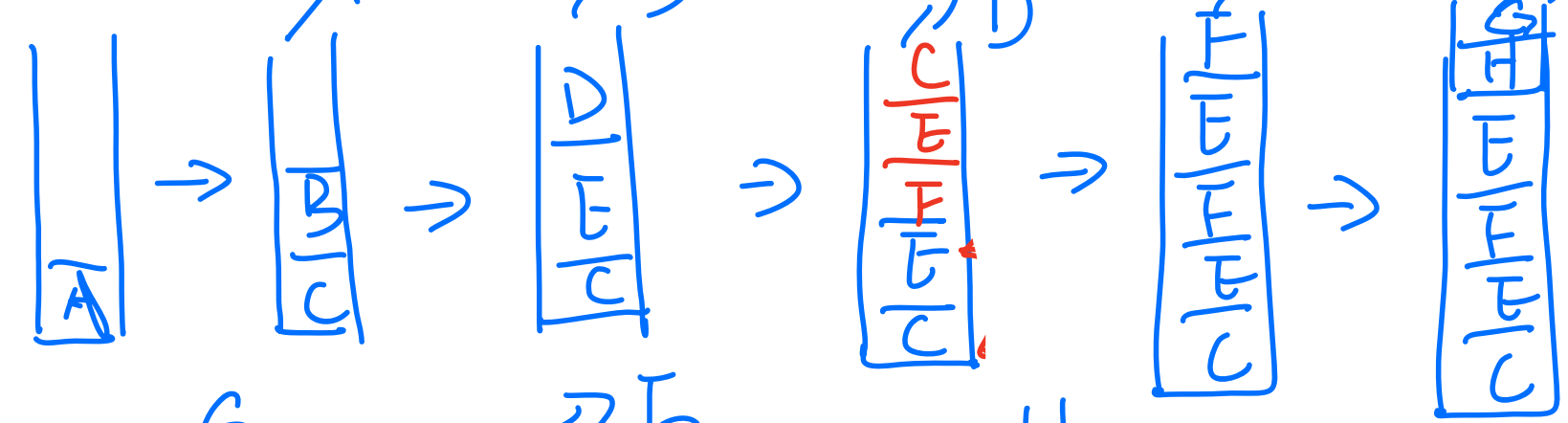
Algorithm:

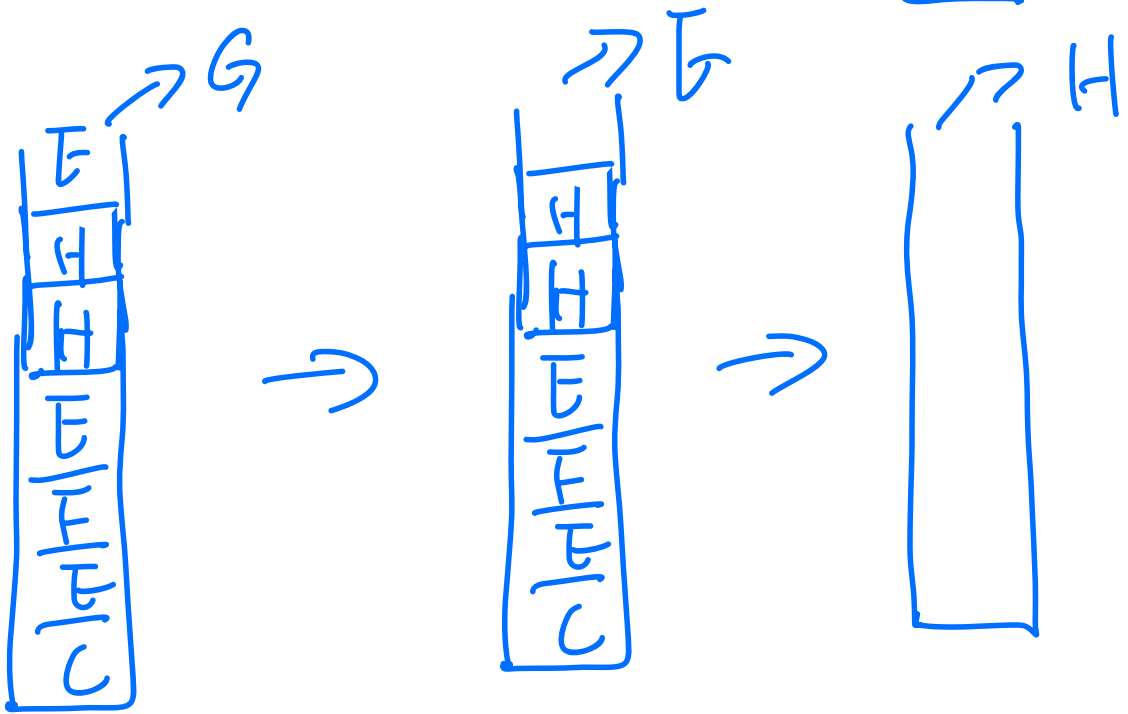
- Initialize a stack with the starting node and an empty path.

- While the stack is not empty:
  - Pop a node and its path from the stack.
  - If the node has been visited, skip it.
  - Mark the node as visited.
  - Append the current node to the path.
  - If the goal is reached, return the path. – Add all unvisited neighbours to the stack.

Deep-First-Search (DFS)







dstar =

badly diretkra dfs

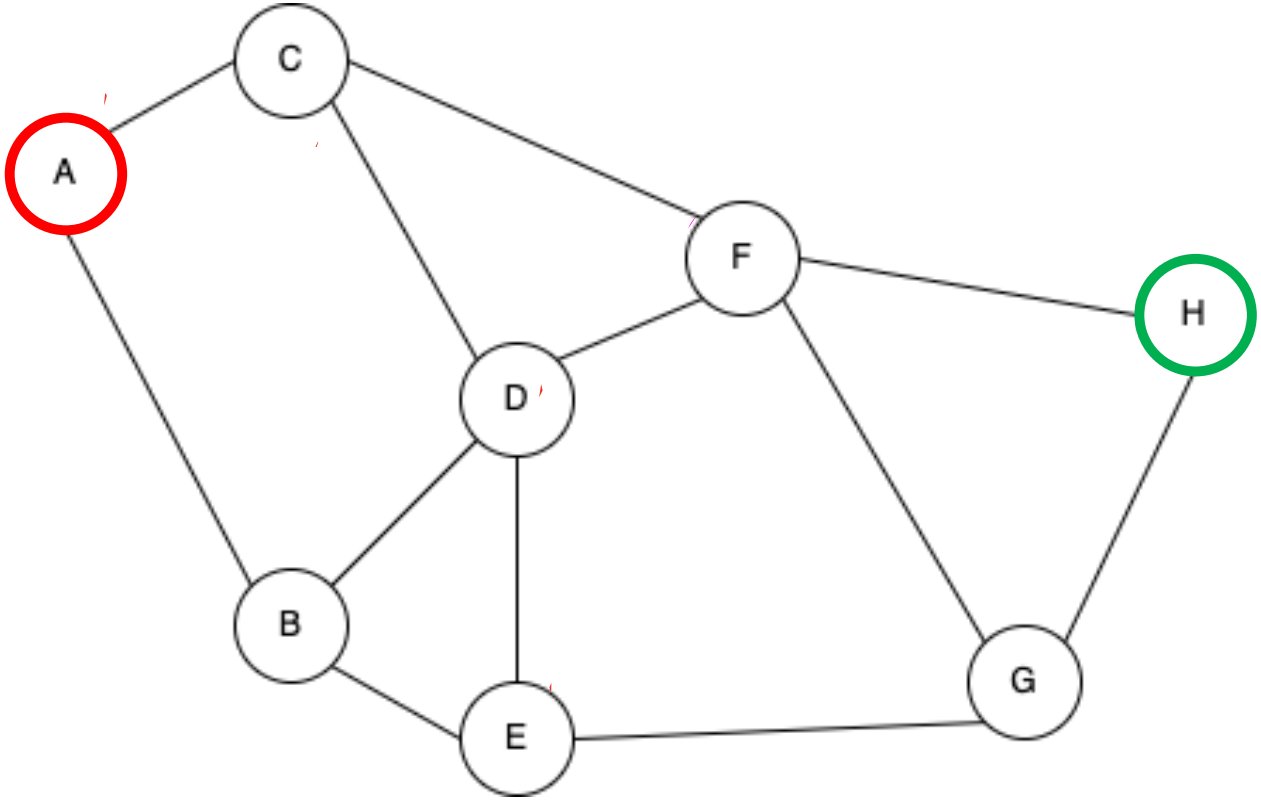
Breadth-First Search (BFS) queue

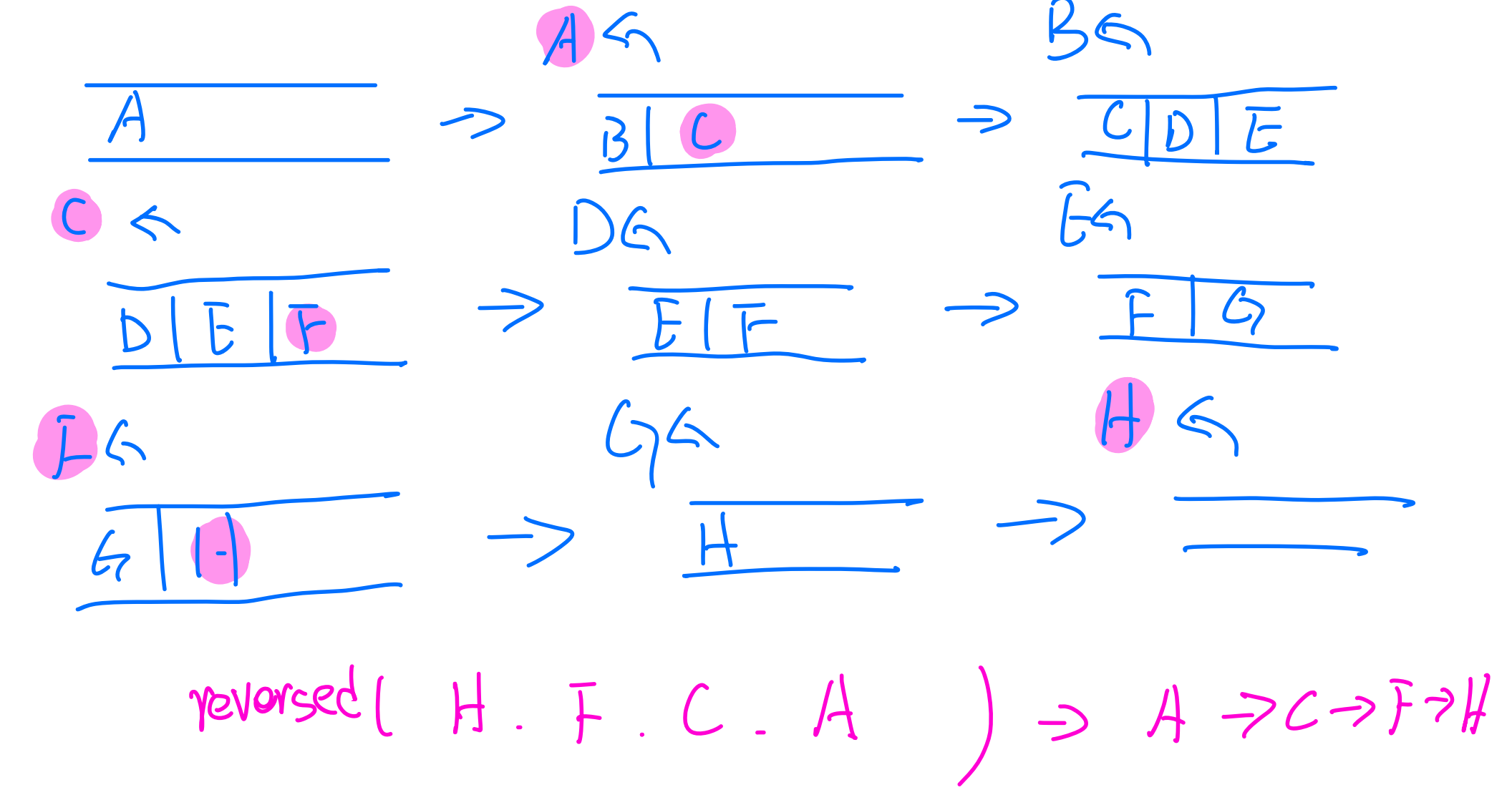
Breadth-First Search (BFS) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root (or some arbitrary node of a graph) and explores the neighbour nodes at the present depth prior to moving on to nodes at the next depth level. BFS uses a queue to keep track of the nodes to be visited next.

Algorithm:

- Initialize a queue with the starting node and an empty path.
- While the queue is not empty:
  - Dequeue a node and its path.
  - If the node has been visited, skip it.
  - Mark the node as visited.
  - Append the current node to the path.
  - If the goal is reached, return the path.
  - Add all unvisited neighbours to the queue.

Breath-First-Search (BFS)





reversed (H . F . C . A) → A → C → F → H

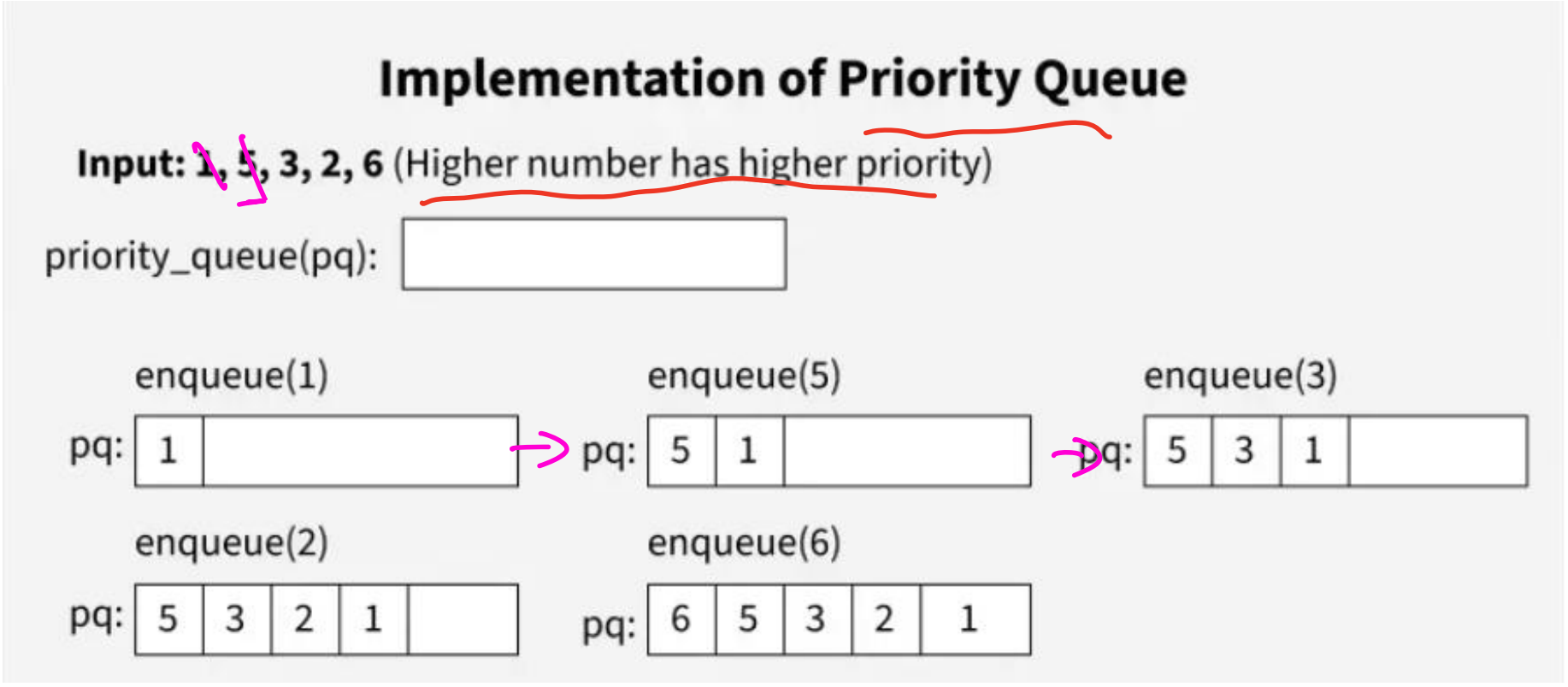
Uniform Cost Search (UCS)

Uniform Cost Search (UCS) is an algorithm used for finding the shortest path in a weighted graph where the cost of edges can vary. It expands the least cost node first, ensuring that the shortest path is found. UCS uses a priority queue to keep track of the nodes to be visited next, with priority given to nodes with the lowest cumulative cost.

Algorithm:

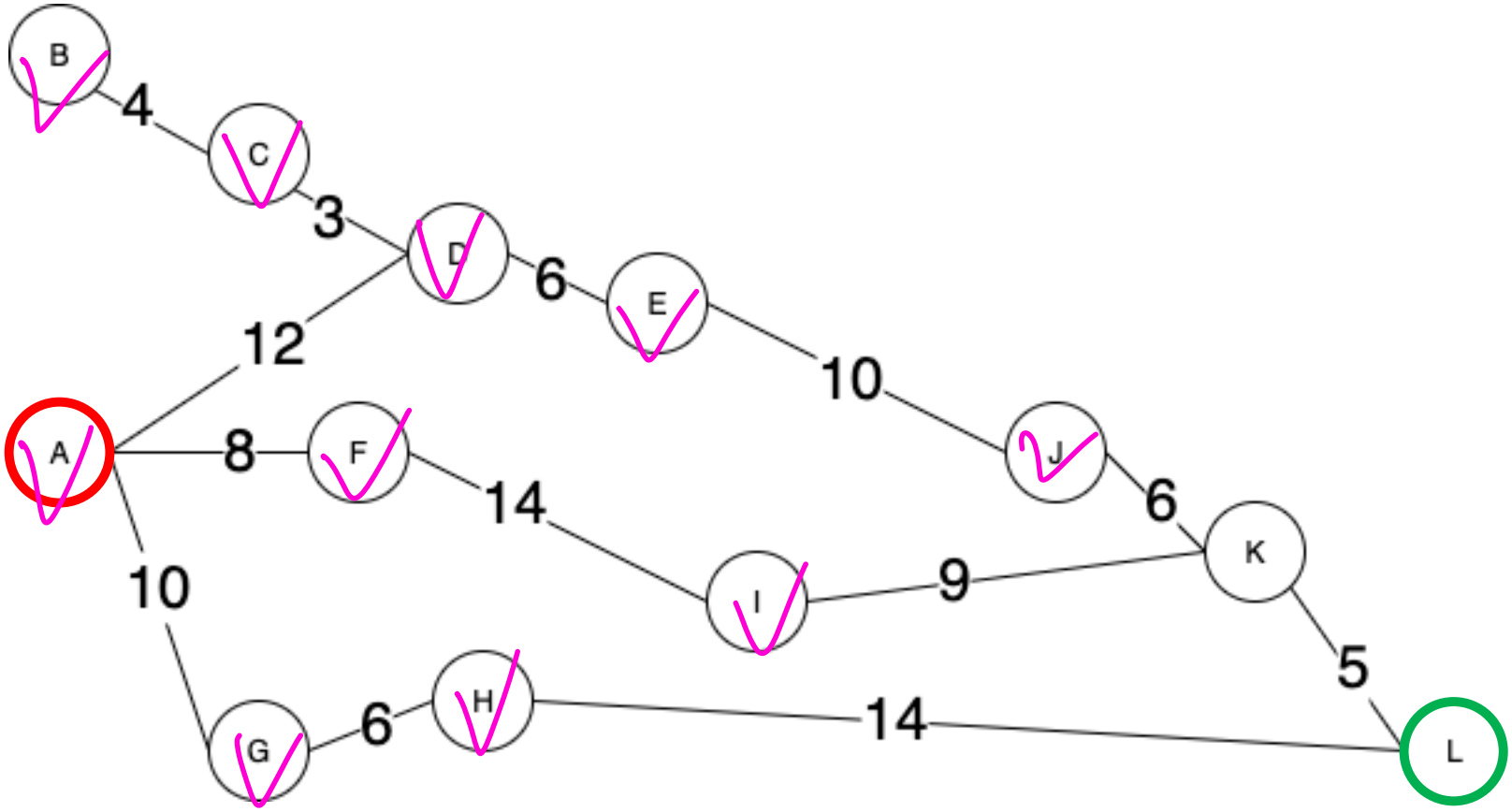
- Initialize a priority queue with the starting node, cost 0, and an empty path.
- While the queue is not empty:
  - Dequeue the node with the lowest cost.
  - If the node has been visited, skip it.
  - Mark the node as visited.
  - Append the current node to the path.
  - If the goal is reached, return the path and cost.
  - Add all unvisited neighbours to the priority queue with their cumulative cost.

Priority queue



Cite: https://www.geeksforgeeks.org/dsa/priority-queue-set-1-introduction/

Uniform Cost Search (UCS)



PQ { A (0) }

A

PQ { F (8) . G (10) . D (12) }

F

PQ { G (10) . D (12) . I (22) }

G

PQ { D (12) . H (16) . I (22) }

D

PQ { C (15) . H (16) . E (18) . I (22) }

PQ^{16} { H(14). E(16). B(19). I(22) }

PQ^{17} { E(16). B(19). I(22). L(30) }

PQ^{15} { B(19). I(22). J(28). L(30) }

PQ^{35} { I(22). J(28). L(30) }

PQ^{16} { J(28) L(30) K(31) }


P Q
{ L (30) k (31) }
L←
P Q
{ }

UNSW SYDNEY

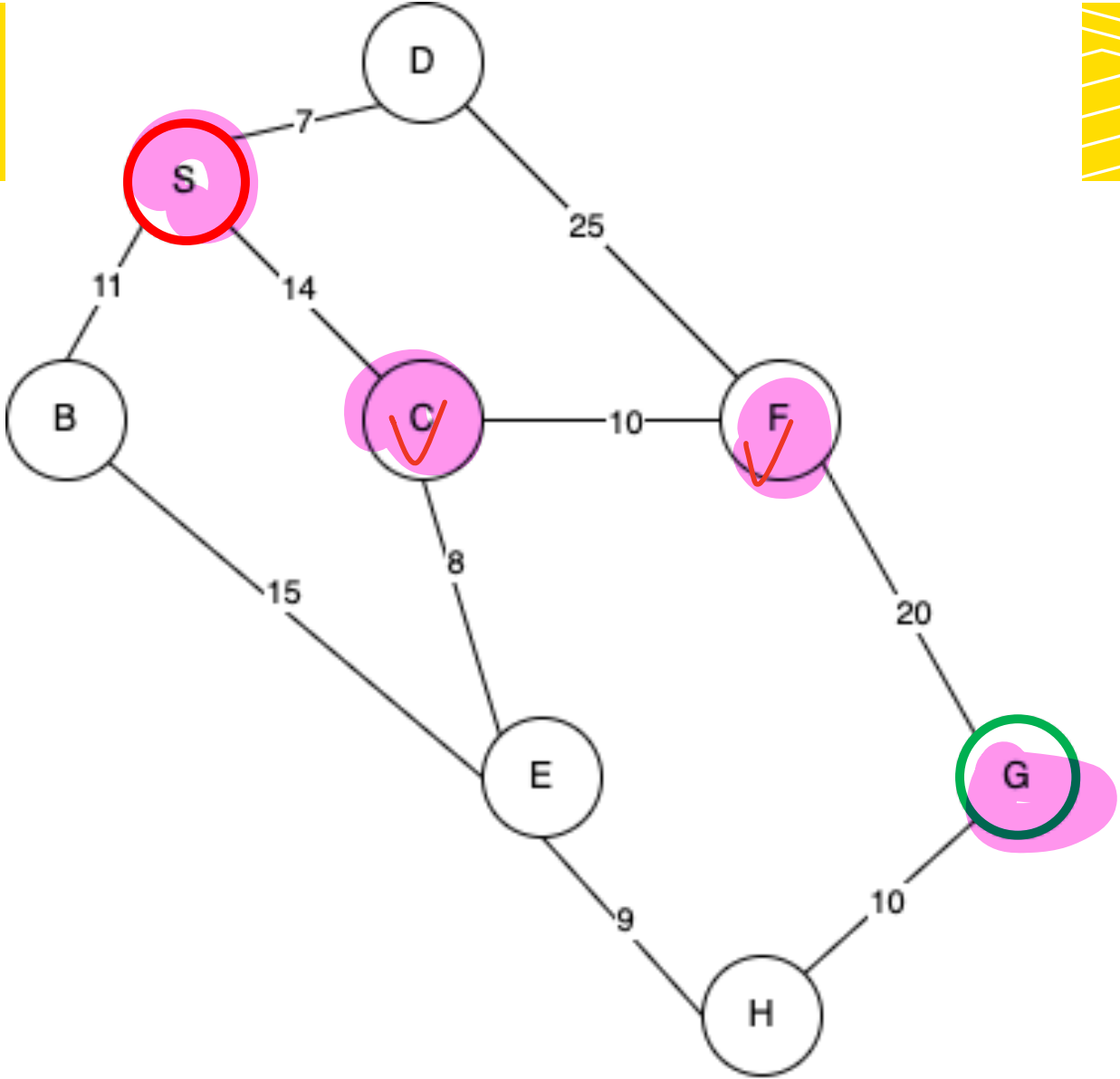
Best-First Search

Best-First Search is an algorithm that uses heuristics to guide the search process. It selects the node that appears to be closest to the goal according to the heuristic function. Best-First Search uses a priority queue to keep track of the nodes to be visited next, with priority given to nodes with the lowest heuristic value.

Algorithm:

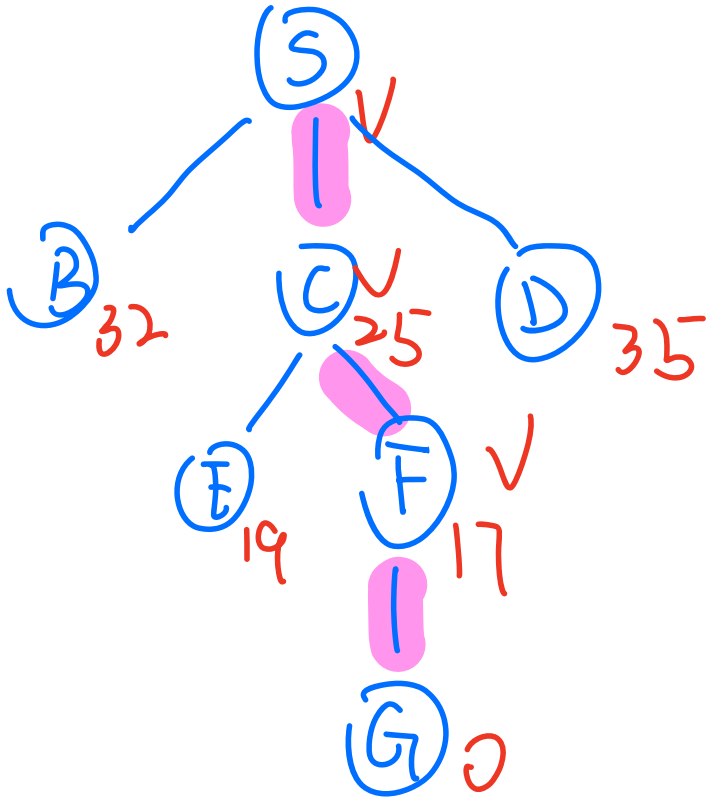
- Initialize a priority queue with the starting node, its heuristic value, and an empty path.
- While the queue is not empty:
  - Dequeue the node with the lowest heuristic value.
  - If the node has been visited, skip it.
  - Mark the node as visited.
  - Append the current node to the path.
  - If the goal is reached, return the path.
  - Add all unvisited neighbours to the priority queue with their heuristic value.

Best First Search



Straight line distance (heuristic)

S -> G = 40
B -> G = 32
C -> G = 25
D -> G = 35
E -> G = 19
F -> G = 17
H -> G = 10
G -> G = 0



Visited

S . C . F

G .

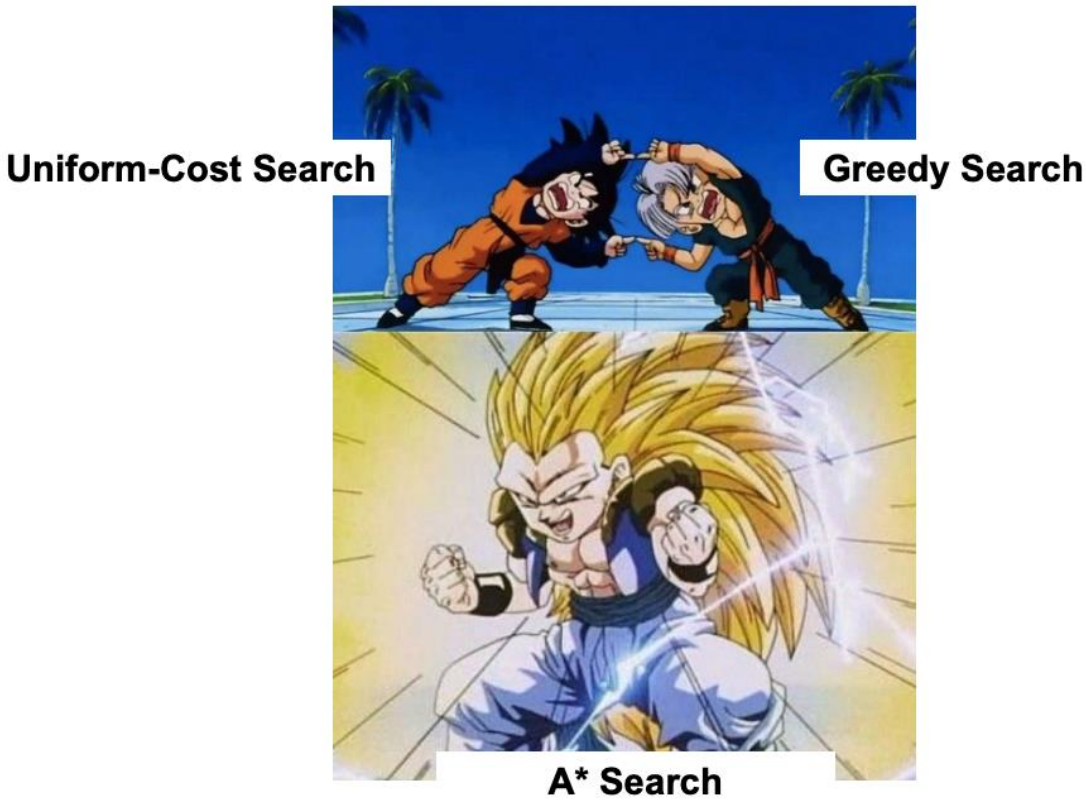
A* Search

A* Search is a widely used algorithm that combines the strengths of Uniform Cost Search and Best-First Search. It uses both the actual cost from the start and the heuristic cost to the goal to guide the search. A* uses a priority queue to keep track of the nodes to be visited next, with priority given to nodes with the lowest combined cost.

Algorithm:

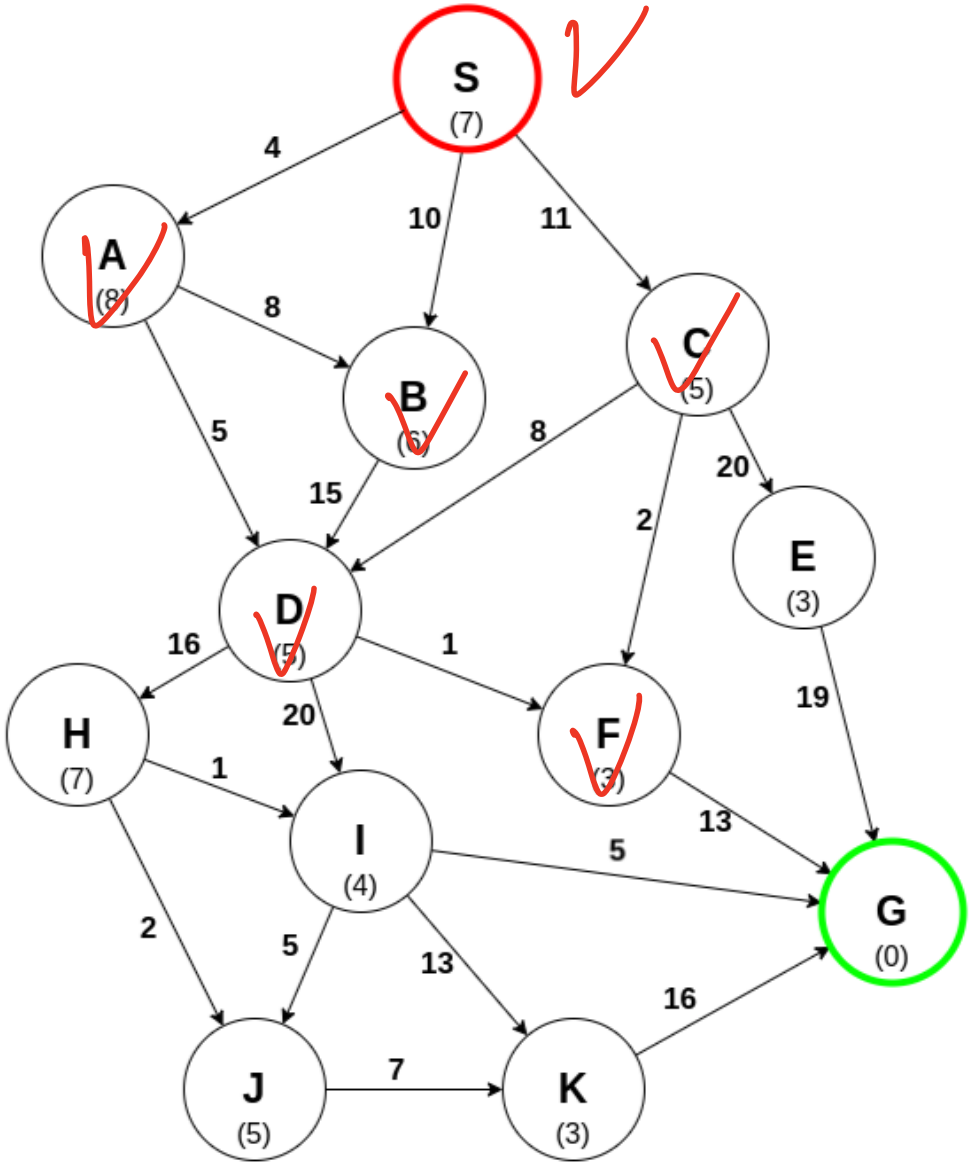
- Initialize a priority queue with the starting node, cost 0, its heuristic value, and an empty path.
- While the queue is not empty:
  - Dequeue the node with the lowest cost + heuristic value.
  - If the node has been visited, skip it.
  - Mark the node as visited.
  - Append the current node to the path.
  - If the goal is reached, return the path and cost.
  - Add all unvisited neighbours to the priority queue with their cumulative cost + heuristic value.

A* Search



Cite: COMP3411/9814 25T3 week1

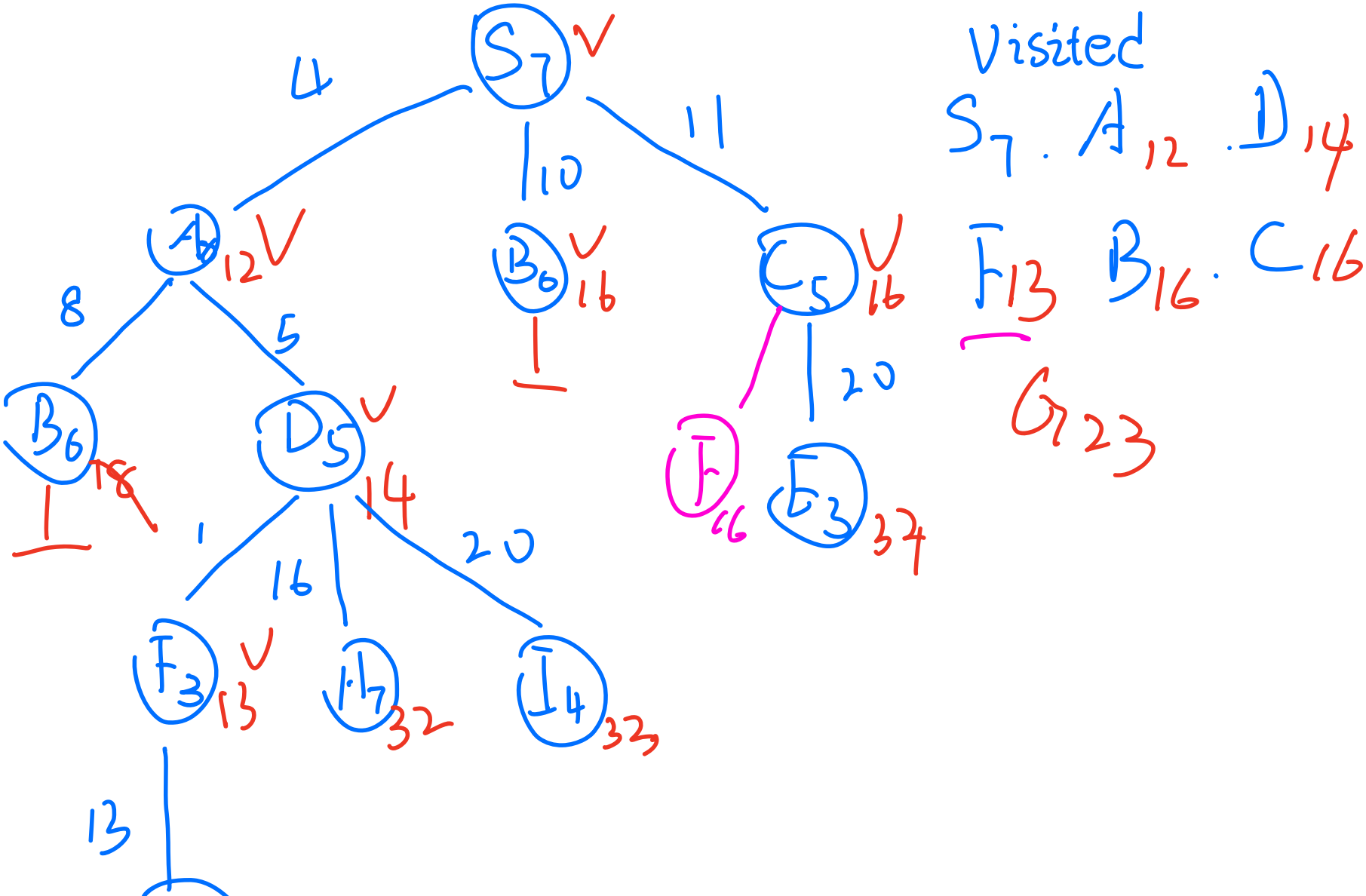
A* Search



Consider the directed graph below. The number on each edge is the actual step cost, and the number in parentheses next to each node is the heuristic value h(n), which estimates the remaining cost from that node to the goal node G.
Run A* search to find the shortest path from the start node S to the goal node G. Assume:

Which path from S to G will A* return?

a. $S \rightarrow C \rightarrow F \rightarrow G$
b. $S \rightarrow C \rightarrow E \rightarrow G$
c. $S \rightarrow B \rightarrow D \rightarrow F \rightarrow G$
d. $S \rightarrow A \rightarrow D \rightarrow F \rightarrow G$
e. $S \rightarrow A \rightarrow D \rightarrow I \rightarrow G$


S7
A12
B6
C5
B6
D5
F
E3
F3
H7
I4
4
10
11
8
5
18
1
14
20
16
20
13
Visited
S7 . A12 . D14
F13 B16 . C16
G23

Artificial Intelligence COMP9414 26T2

M11A, H18A tutorial

08/06/2026 Monday

~Questions and Suggestions~

Thanks for listening

Joffrey Ji (z5450981@ad.unsw.edu.au)